CHAPTER 2

ARRAYS AND STRUCTURES

All the programs in this file are selected from

Ellis Horowitz, Sartaj Sahni, and Susan Anderson-Freed
"Fundamentals of Data Structures in C",

Arrays (陣列)

Array: a set of index and value (一個索引和值的集合)

data structure

For each index, there is a value associated with that index. (每一個索引都會連接到一個值)

representation (possible)

implemented by using consecutive memory (連續記憶體位置).

Structure *Array* is

objects: A set of pairs <*index*, *value*> where for each value of *index* there is a value from the set *item* (每個index都有來自集合item的對應值). *Index* is a finite ordered set of one or more dimensions (一維或多維的有限序集), for example, {0, … , n-1} for one dimension (一維), {(0,0), (0,1), (0,2), (1,0), (1,1), (1,2), (2,0), (2,1), (2,2)} for two dimensions (二維), etc.

Functions:

for all A ∈ Array, *i* ∈ *index*, x ∈ *item*, *j*, *size* ∈ integer

Array Create(j, list) ::= **return** an array of *j* dimensions (一個j維陣列) where list is a j-tuple (list是一個j元序對) whose *i*th element is the size of the *i*th dimension (第i個元素是第i維的大小). *Items* are undefined. (Items尚未定義)

Item Retrieve(A, *i*) ::= **if** (*i* ∈ *index*) **return** the item associated with index value *i* in array A (A陣列裡索引值為i的關聯項目) **else return** error

Array Store(A, *i*, *x*) ::= **if** (*i* in *index*) **return** an array that is identical to array A except the new pair <*i*, *x*> has been inserted (一個和A一樣的陣列,除了新插入的一對<*i*, *x*>) **else return** error

end array

*__Structure 2.1:__ Abstract Data Type *Array* (p.52)

Arrays in C

int list[5], *plist[5];

list[5]: five integers
list[0], list[1], list[2], list[3], list[4]
*plist[5]: five pointers to integers
plist[0], plist[1], plist[2], plist[3], plist[4]

implementation of 1-D array

list[0]	base address = α
list[1]	α + sizeof(int)
list[2]	α + 2*sizeof(int)
list[3]	α + 3*sizeof(int)
list[4]	α + 4*size(int)

Arrays in C (*Continued*)

Compare int *list1 and int list2[5] in C.

Same: list1 and list2 are pointers.
Difference: list2 reserves five locations.

Notations:

list2 - a pointer to list2[0]
(list2 + i) - a pointer to list2[i] (&list2[i])
*(list2 + i) - list2[i]

Example: 1-dimension array addressing

int one[] = {0, 1, 2, 3, 4};
Goal: print out address and value

```
void print1(int *ptr, int rows)
{
/* print out a one-dimensional array using a pointer */
        int i;
        printf("Address Contents\n");
        for (i=0; i < rows; i++)
                printf("%8u%5d\n", ptr+i, *(ptr+i));
        printf("\n");
}
```

call print1(&one[0], 5)

Address	Contents
1228	0
1230	1
1232	2
1234	3
1236	4

*__Figure 2.1:__ One- dimensional array addressing (p.55)

Structures (結構) (records)

```
struct {
        char name[10];
        int age;
        float salary;
        } person;

strcpy(person.name, “james”); // copy one string into another
person.age=10;
person.salary=35000;
```

Create structure data type

```
typedef struct human_being {
        char name[10];
        int age;
        float salary;
        };

or

typedef struct {
        char name[10];
        int age;
        float salary
        } human_being;

human_being person1, person2;
```

Unions (聯合)

Similar to struct, but only one field is active (只有一個欄位是有作用的).

Example: Add fields for male and female.

```
typedef struct sex_type {
        enum tag_field {female, male} sex;
        union {
                int children;
                int beard;
                } u;
        };
typedef struct human_being {
        char name[10];
        int age;
        float salary;
        date dob;
        sex_type sex_info;
        }
```

```
human_being person1, person2;
person1.sex_info.sex=male;
person1.sex_info.u.beard=FALSE;
```

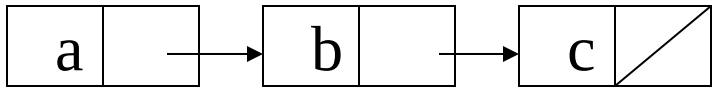
Self-Referential Structures (自我參考結構)

One or more of its components is a pointer to itself. (裡面的成員有一個或多個是指向自己的指標)

```
typedef struct list {
        char data;
        list *link;
        }
```

Construct a list with three nodes
item1.link=&item2;
item2.link=&item3;
malloc: obtain a node

```
list item1, item2, item3;
item1.data='a';
item2.data='b';
item3.data='c';
item1.link=item2.link=item3.link=NULL;
```



Ordered List (序列) Examples

ordered (linear) list: (item1, item2, item3, …, item*n*)

(MONDAY, TUEDSAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAYY, SUNDAY)

(2, 3, 4, 5, 6, 7, 8, 9, 10, Jack, Queen, King, Ace)

(1941, 1942, 1943, 1944, 1945)

(a_1, a_2, a_3, …, a_{n-1}, a_n)

Operations (運算) on Ordered List

Find the length, n , of the list.

Read the items from left to right (or right to left).

Retrieve the i'th element.

Store a new value into the i'th position.

Insert a new element at the position i , causing elements numbered i, i+1, …, n to become numbered i+1, i+2, …, n+1

Delete the element at position i , causing elements numbered i+1, …, n to become numbered i, i+1, …, n-1

Polynomials $A(X)=3X^{20}+2X^{5}+4$, $B(X)=X^{4}+10X^{3}+3X^{2}+1$

Structure *Polynomial* is

objects: $p(x)=a_1x^{e_1}+\ldots+a_nx^{e_n}$; a set of ordered pairs (成對序集) of $<e_i,a_i>$ where a_i in *Coefficients* (係數) and e_i in *Exponents* (指數), e_i are integers >= 0

functions:

for all *poly*, *poly1*, *poly2* ∈ *Polynomial*, *coef* ∈ *Coefficients*, *expon* ∈ *Exponents*

Polynomial Zero( ) ::= **return** the polynomial, $p(x) = 0$

Boolean IsZero(*poly*) ::= **if** (*poly*) **return** *FALSE* **else return** *TRUE*

Coefficient Coef(*poly, expon*) ::= **if** (*expon* ∈ *poly*) **return** its coefficient **else return** Zero

Exponent Lead_Exp(*poly*) ::= **return** the largest exponent (最高次方) in *poly*

Polynomial Attach(*poly,coef, expon*) ::= **if** (*expon* ∈ *poly*) **return** error **else return** the polynomial poly with the term <*coef, expon*> inserted (插入<*coef, expon*>後的多項式poly)

Polynomial Remove(*poly, expon*) ::= **if** (*expon* ∈ *poly*) **return** the polynomial *poly* with the term whose exponent is *expon deleted* (刪除次方是*expon*的項後的多項式*poly*)
else return error

Polynomial SingleMult(*poly, coef, expon*) ::= **return** the polynomial *poly* • *coef* • x^{expon}

Polynomial Add(*poly1, poly2*) ::= **return** the polynomial *poly*1 +*poly*2

Polynomial Mult(*poly1, poly2*) ::= **return** the polynomial *poly*1 • *poly*2

End *Polynomial*

***Structure 2.2:** Abstract data type *Polynomial* (p.67)

Polynomial Addition (多項式加法)

data structure 1:

```
#define MAX_DEGREE 101
typedef struct {
        int degree;
        float coef[MAX_DEGREE];
        } polynomial;
```

```
/* d =a + b, where a, b, and d are polynomials */
d = Zero( )
while (! IsZero(a) && ! IsZero(b)) do {
   switch COMPARE (Lead_Exp(a), Lead_Exp(b))  {
       case -1 : d =
           Attach(d, Coef (b, Lead_Exp(b)), Lead_Exp(b));
           b = Remove(b, Lead_Exp(b));
           break;
      case  0 : sum = Coef (a, Lead_Exp (a)) + Coef (b, Lead_Exp(b));
          if (sum) {
              Attach (d, sum, Lead_Exp(a));
              a = Remove(a , Lead_Exp(a));
              b = Remove(b , Lead_Exp(b));
              }
           break;
```

```
case 1 : d =
            Attach(d, Coef (a, Lead_Exp(a)), Lead_Exp(a));
            a = Remove(a, Lead_Exp(a));
         }
    }
insert any remaining terms of a or b into d
```

advantage: easy implementation

disadvantage: waste space when sparse

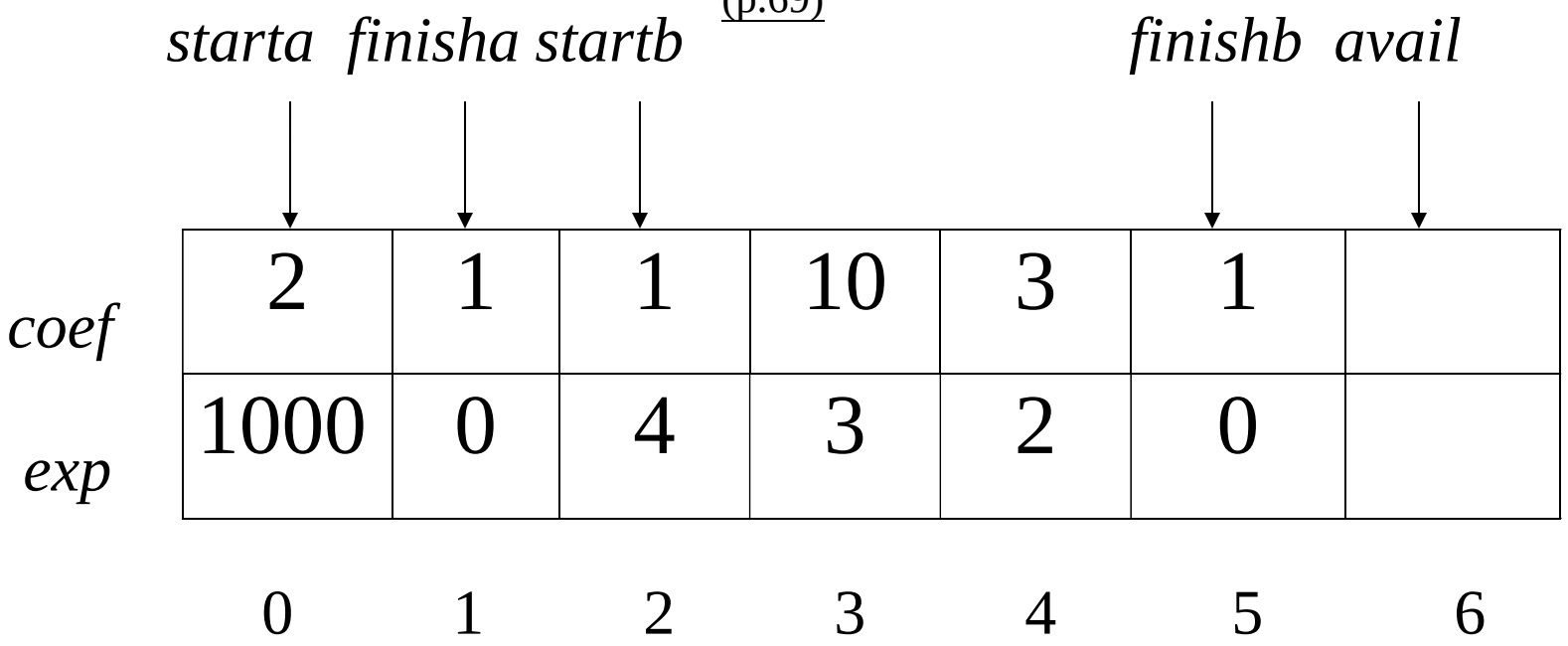
***Program 2.5:** Initial version of *padd* function(p.68)

Data structure 2: use one global array to store all polynomials

$A(X)=2X^{1000}+1$

$B(X)=X^4+10X^3+3X^2+1$

***Figure 2.3:** Array representation of two polynomials (p.69)



specification	representation
poly	<start, finish>
A	<0,1>
B	<2,5>

storage requirements: start, finish, 2*(finish-start+1)
nonsparse: twice as much as (1)
when all the items are nonzero

```
MAX_TERMS 100 /* size of terms array */
typedef struct {
        float coef;
        int expon;
        } polynomial;
polynomial terms[MAX_TERMS];
int avail = 0;
```

*(p.68)

Add two polynomials: D = A + B

```
void padd (int starta, int finisha, int startb, int finishb,
                                        int * startd, int *finishd)
{
/* add A(x) and B(x) to obtain D(x) */
    float coefficient;
   *startd = avail;
   while (starta <= finisha && startb <= finishb)
     switch (COMPARE(terms[starta].expon,
                                terms[startb].expon)) {
       case -1: /* a expon < b expon */
              attach(terms[startb].coef, terms[startb].expon);
              startb++
              break;
```

```
case  0: /* equal exponents */
            coefficient = terms[starta].coef +
                                terms[startb].coef;
            if (coefficient)
               attach (coefficient, terms[starta].expon);
            starta++;
            startb++;
            break;
case 1: /* a expon > b expon */
         attach(terms[starta].coef, terms[starta].expon);
         starta++;
}
```

```
/* add in remaining terms of  A(x) */
for( ; starta <= finisha; starta++)
     attach(terms[starta].coef, terms[starta].expon);
/* add in remaining terms of B(x) */
for( ; startb <= finishb; startb++)
    attach(terms[startb].coef, terms[startb].expon);
*finishd =avail -1;
}
```

Analysis: O(n+m)

where n (m) is the number of nonzeros in A(B).

***Program 2.6:** Function to add two polynomial (p.70)

```
void attach(float coefficient, int exponent)
{
/* add a new term to the polynomial */
    if (avail >= MAX_TERMS) {
       fprintf(stderr, “Too many terms in the polynomial\n”);
       exit(1);
      }
      terms[avail].coef  = coefficient;
      terms[avail++].expon = exponent;
}
```

***Program 2.7:** Function to add anew term (p.71)

Problem: Compaction (壓縮) is required when polynomials that are no longer needed. (data movement takes time.)

Sparse Matrix (稀疏矩陣)

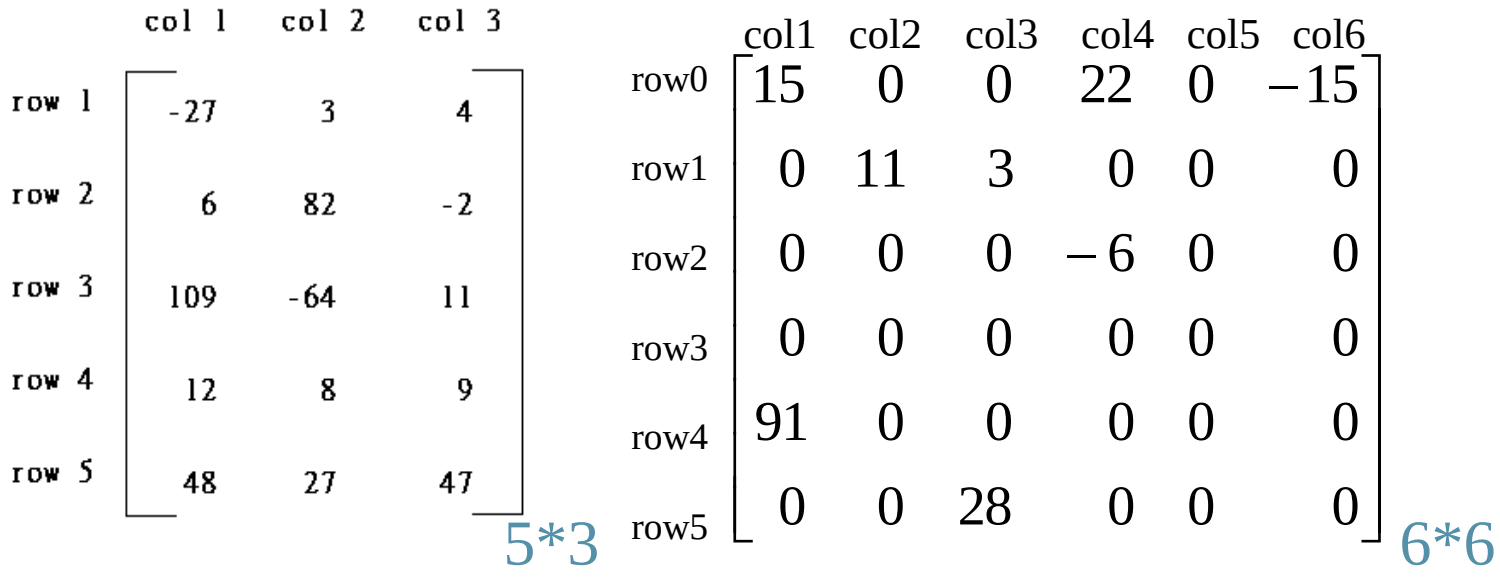


***Figure 2.4: Two matrices**

SPARSE MATRIX ABSTRACT DATA TYPE

Structure *Sparse_Matrix* is

objects: a set of triples, <*row*, *column*, *value*>, where *row* and *column* are integers and form a unique combination, and *value* comes from the set *item*.

functions:

for all *a*, *b* ∈ *Sparse_Matrix*, *x* ∈ *item*, *i*, *j*, *max_col*, *max_row* ∈ *index*

Sparse_Marix Create(*max_row*, *max_col*) ::=

return a *Sparse_matrix* that can hold up to *max_items* = *max _row* x *max_col* and whose maximum row size is *max_row* and whose maximum column size is *max_col*.

Sparse_Matrix Transpose(*a*) ::=

return the matrix produced by interchanging the row and column value (列和行的值交換) of every triple.

Sparse_Matrix Add(*a*, *b*) ::=

if the dimensions of a and b are the same (a和b同維度) **return** the matrix produced by adding corresponding items, namely those with identical *row* and *column* values.

else return error

Sparse_Matrix Multiply(*a*, *b*) ::=

if number of columns in a equals number of rows in b (a的行數和b的列數相等)

return the matrix *d* produced by multiplying a by b according to the formula: *d* [*i*] [*j*] = Σ (a[i][k]•b[k][j]) where *d (i, j)* is the *(i,j)*th element

else return error.

*** Structure 2.3:** Abstract data type Sparse-Matrix (p.74)

(1) Represented by a two-dimensional array.
Sparse matrix wastes space.

(2) Each element is characterized by <row, col, value>.

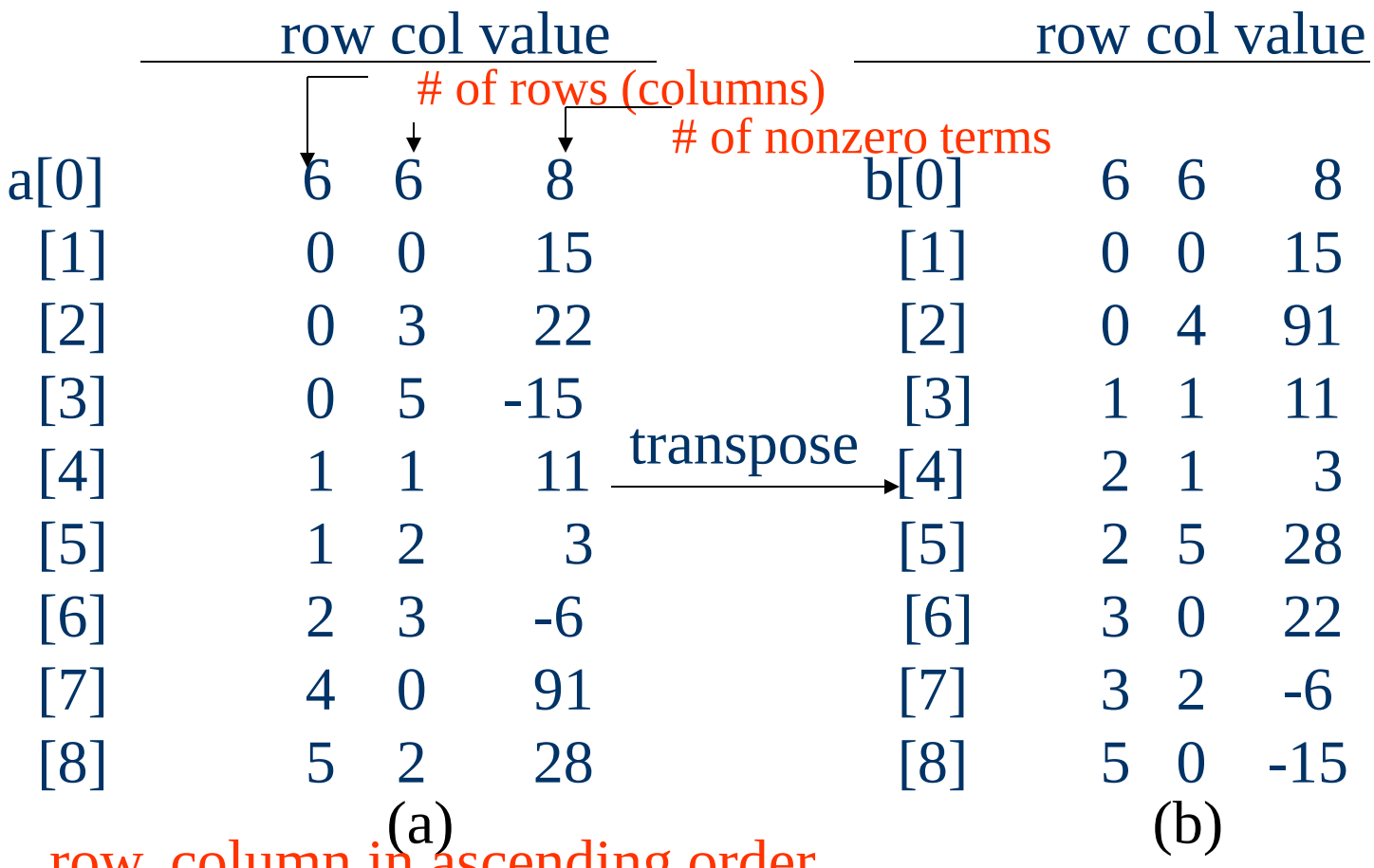


***Figure 2.5:** Sparse matrix and its transpose (轉置) stored as triples (三元序對) (p.75)

Sparse_matrix Create(max_row, max_col) ::=

```
#define MAX_TERMS 101 /* maximum number of terms +1*/
    typedef struct {
                int col;
                int row;
                int value;
                } term;
    term a[MAX_TERMS]
```

* (P.75)

Transpose (轉置) a Matrix

(1) for each row i

take element <i, j, value> and store it

in element <j, i, value> of the transpose.

difficulty: where to put <j, i, value>

(0, 0, 15) ====> (0, 0, 15)

(0, 3, 22) ====> (3, 0, 22)

(0, 5, -15) ====> (5, 0, -15)

(1, 1, 11) ====> (1, 1, 11)

Move elements down very often.

(2) For all elements in column j,

place element <i, j, value> in element <j, i, value>

```
void transpose (term a[], term b[])
/* b is set to the transpose of a */
{
   int n, i, j, currentb;
   n = a[0].value;  /* total number of elements */
   b[0].row = a[0].col;  /* rows in b = columns in a */
   b[0].col = a[0].row;  /*columns in b = rows in a */
   b[0].value = n;
   if (n > 0) {                /*non zero matrix */
      currentb = 1;
      for (i = 0; i < a[0].col; i++)
      /* transpose by columns in a */
         for( j = 1; j <=  n; j++)
         /*  find elements from the current column */
            if (a[j].col == i) {
            /* element is in current column, add it to b */
```

```
columns
    elements
        b[currentb].row = a[j].col;
        b[currentb].col  = a[j].row;
        b[currentb].value = a[j].value;
        currentb++
    }
  }
}
```

*** Program 2.8:** Transpose of a sparse matrix (p.77)

Scan the array "columns" times.
The array has "elements" elements. ==> O(columns*elements)

Discussion: compared with 2-D array representation

O(columns*elements) vs. O(columns*rows)

elements --> columns * rows when nonsparse
O(columns*columns*rows)

Problem: Scan the array “columns” times.

Solution:

Determine the number of elements in each column of the original matrix.
==>
Determine the starting positions of each row in the transpose matrix.

a[0]	6	6	8
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	-15
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28

	[0]	[1]	[2]	[3]	[4]	[5]
row_terms =	2	1	2	2	0	1
starting_pos =	1	3	4	6	8	8

```
void fast_transpose(term a[ ], term b[ ])
{
/* the transpose of a is placed in b */
  int row_terms[MAX_COL], starting_pos[MAX_COL];
  int i, j, num_cols = a[0].col, num_terms = a[0].value;
  b[0].row = num_cols; b[0].col = a[0].row;
  b[0].value = num_terms;
  if (num_terms > 0){ /*nonzero matrix*/
    for (i = 0; i < num_cols; i++)
        row_terms[i] = 0;
    for (i = 1; i  <= num_terms; i++)
        row_term [a[i].col]++
    starting_pos[0] = 1;
    for (i =1; i < num_cols; i++)
        starting_pos[i]=starting_pos[i-1] +row_terms [i-1];
```

columns

elements

columns

```
                for (i=1; i <= num_terms, i++) {
                        j = starting_pos[a[i].col]++;
elements                b[j].row = a[i].col;
                        b[j].col = a[i].row;
                        b[j].value = a[i].value;
                }
        }
}
```

***Program 2.9:** Fast transpose of a sparse matrix (快速的轉置稀疏矩陣)

Compared with 2-D array representation

O(columns+elements) vs. O(columns*rows)

elements --> columns * rows

O(columns+elements) --> O(columns*rows)

Cost: Additional row_terms and starting_pos arrays are required.

Let the two arrays row_terms and starting_pos be shared.